CIS

Critical Infrastructure Security

Software Requirements

- Docker Desktop - https://www.docker.com/products/docker-desktop/

Usage

- First, pull the repository into your file system

```
$  git clone https://github.com/edogdu/CIS.git ./CIS
```

- Next, switch to root directory of project

```
$  cd CIS
```

Base Application Startup

Base application startup is a requirement for each of the other modules. This should be started first, everytime.

```
$ docker-compose up -d cis-kafka cis-neo4j cis-timescaledb cis-test-data-loader
```

Initial Setup

Run these once for initial data load. Base Application should be up first.

```
$ docker-compose up -d cis-init-timescaledb

$ docker-compose up -d cis-spark-master cis-spark-worker cis-kafka-init-topics
cis-raw-network-consumer cis-raw-physical-consumer cis-data-error-consumer

$ docker-compose up -d cis-load-bigdata
```

Test Data Loader FastAPI

The remaining sections utilize a FastAPI test harness to act as a trigger for various events. With the Base Application started, you can view the full list of available functions and expected payloads here:

http://localhost:8090/docs

The functions are listed in order of completion. It is assumed that you have run each subsequent step before performing the next one. It is also assumed that the Base application is started.

Perform Data Aggregation

Perform Http GET to the following url:

http://localhost:8090/refresh_materialized_views

Generate Graphs

Ensure the following Docker containers are started and ready

```
$ docker-compose up -d cis-graph-generator-consumer
```

Perform Http POST to the following url:

http://localhost:8090/generate_graph

```
{
  "start_time": "2020-01-01T03:54:48.658Z",
  "end_time": "2025-01-01T03:54:48.658Z",
  "system_id": "testbed_system_1",
  "duration": 30
}
```

Train Models

Ensure the following Docker container is started and ready

```
$ docker-compose up -d cis-anomaly-detector-consumer
```

Perform Http POST to the following url:

http://localhost:8090/train_gnn_model

```
{
  "snapshot_id": "",
  "start_time": "2020-04-09T18:23:00+00",
  "end_time": "2025-04-09T18:23:00+00",
  "duration": 30,
  "system_id": "testbed_system_1",
  "edge_threshold_percent": 0.001,
  "snapshot_threshold_percent": 0.90,
  "model_type": "1",
  "xai_type": "3",
```

```
  "is_train": true,
  "export_model": true,
  "export_performance": true,
  "is_threshold_only": false
}
```

- NOTE: Currently, model parameters are not connected to the configuration passed into our model. That will need to be setup in /src/anomalies/anomaly_consumer.py

UI Dashboard

Ensure the following Docker containers are started and ready

```
$ docker-compose up -d cis-dashboard-app
```

Navigate to the following URL:

http://localhost:8050/

Troubleshooting

Notes about Ports

- The following have had their ports mapped to non-standard ports to avoid collision with UCKG
  - **Neo4j**
    - 7485 for web ui
    - 7698 for bolt
  - **Postgres**
    - 5445

Notes about Configuring Spark and Kafka

Depending on the hosting machine's available resources, the following services in docker-compose.yml may need to be adjusted accordingly.

- Spark - https://spark.apache.org/docs/latest/hardware-provisioning.html
- Kafka - https://docs.redhat.com/en/documentation/red_hat_streams_for_apache_kafka/2.9/html/kafka_configuration_tuning/con-broker-config-properties-str#improving_request_handling_throughput_by_increasing_i_o_threads

```
cis-spark-worker:
    SPARK_WORKER_MEMORY: "16G"
    SPARK_WORKER_CORES: "8"

cis-load-bigdata:
    entrypoint:
    - "spark.executor.instances=2"
```

```
      - "--conf"
      - "spark.executor.memory=8g"
      - "--conf"
      - "spark.executor.cores=4"
      - "--conf"
      - "spark.driver.memory=8g"

  cis-kafka:
      environment:
          KAFKA_HEAP_OPTIONS: "-Xms6g -Xmx6g"
          KAFKA_CFG_NUM_NETWORK_THREADS: "6"
          KAFKA_CFG_NUM_IO_THREADS: "16"
          KAFKA_CFG_SOCKET_SEND_BUFFER_BYTES: "1048576"
          KAFKA_CFG_MESSAGE_MAX_BYTES: "2000000"
          KAFKA_CFG_REPLICA_SOCKET_RECEIVE_BUFFER_BYTES: "1048576"
```

Resources

- testbed_system_1 assumes that you have downloaded A hardware-in-the-loop water distribution testbed (WDT) dataset for cyber-physical security testing dataset from IEEE Dataport. See /data/testbed_system_1/CITATIONS.md for more information.
- Docker has a tendency to have hanging resources that can take up alot of diskspace, I found the following commands useful
    - In Docker Desktop Application, ensure no containers are running
    - In Command Line or Bash run Docker prune commands

```
$  docker image prune -f
```

```
$  docker builder prune
```

    - In Docker Desktop Application, navigate to Troubleshoot section (bug button) and select Clean/Purge data
    - If diskspace utilization is not changing after running these steps, try restarting your computer

License